

02321 Hardware/software

Fall 2026

Digital Watch Emulator – Casio F-91W

Authors

Group 6



Gerdorf, Ludvig H.
s255479@student.dtu.dk



Jensen, Frederik B.
s242327@student.dtu.dk

Contents

1 Contributions **3**

2 Requirements **4**
2.1 Need to have . . . 4
2.2 Nice to have . . . 4

3 Analysis **5**
3.1 Overall idea . . . 5
3.2 Interrupts for buttons and switches . . . 5
3.3 Button bounce . . . 5
3.4 ASCII graphics . . . 5
3.5 Cycling between modes . . . 5
3.6 Setting the time . . . 5
3.7 Quick digit advance . . . 6
3.8 Showing the time and speeding it up . . . 6
3.9 Stopwatch with net time measurement . . . 6
3.10 Alarm . . . 6
3.11 Day and date . . . 6

4 Risk Analysis **7**

5 Design **8**
5.1 Hardware overview . . . 8
5.2 Software structure . . . 8
5.3 Main loop . . . 9
5.4 Interrupts . . . 9
5.5 Mode changer . . . 10
5.6 Stopwatch . . . 11
5.7 Setting and quick digit advance . . . 11
5.8 Speed-up . . . 12
5.9 Alarm . . . 12
5.10 Day and date . . . 12
5.11 Display design . . . 12

6 Implementation **14**
6.1 Hardware setup and interrupts . . . 14
6.2 Main loop . . . 14
6.3 Button interrupts . . . 14
6.4 Switch interrupt . . . 14
6.5 Clock, setting and quick digit advance . . . 14
6.6 Stopwatch . . . 15
6.7 ASCII graphics . . . 15
6.8 Alarm . . . 15
6.9 Day and date . . . 15

7 Test **17**
7.1 Test method . . . 17
7.2 Test summary . . . 17

8 Discussion **19**
8.1 Fulfilment of the requirements . . . 19

8.2 What worked well 19
8.3 Limitations 19
8.4 Comparison with the real Casio F-91W 20
8.5 Future work 20

9 Conclusion **21**

A User manual **22**

B Bug report **23**

C Test reports **24**

D Extra diagrams **25**

1 Contributions

Code	*Name*
Hardware design (Vivado)	Frederik & Ludvig
Interrupts	Frederik
ASCII graphics	Ludvig
Cycle between modes	Ludvig
Set the time	Frederik
Quick digit advance	Frederik
Show time	Ludvig & Frederik
Switch speed-up and priority	Frederik
Full stopwatch	Frederik
Alarm	Ludvig
Day and date	Ludvig

Table 1: Primary area of responsibility for the code. Note that all members collaborated across all parts of the project.

Report	*Name*
Contribution	Frederik & Ludvig
Requirements	Frederik
Analysis	Ludvig & Frederik
Risk analysis	Ludvig
Design	Frederik & Ludvig
Implementation	Frederik & Ludvig
Tests	Frederik & Ludvig
Discussion	Frederik & Ludvig
Conclusion	Frederik
Appendix	Ludvig

Table 2: Primary area of responsibility for the report. Note that all members collaborated across all parts of the project.

AI has been used to proofread and make the report more readable.

2 Requirements

The goal of the project is to emulate the Casio F-91W digital watch on the Zybo board. The buttons and switches on the board are the inputs, and the watch face is drawn with ASCII graphics in a serial terminal. Each requirement has an ID, so it can be traced to the tests in Section 7 and to the discussion in Section 8.1.

2.1 Need to have

R1 Buttons and switches are handled with interrupts (no polling).

R2 Time values are shown with ASCII graphics. The image is static on the screen: no scrolling and no visible flicker.

R3 Button C cycles through the watch modes in the order Time → Alarm → Stopwatch → Setting → Time. The current mode is always shown.

R4 The time (hours, minutes and seconds) can be set.

R5 Quick digit advance: holding button A for more than 2 seconds in Setting mode makes the selected value advance automatically until A is released.

R6 The time is shown and can be sped up with the switches: SW0 = 10x, SW1 = 20x, SW2 = 30x and SW3 = 60x. If more switches are on, the highest switch decides.

R7 Stopwatch with a resolution of 1/100 second, start/stop, reset and net time measurement.

2.2 Nice to have

N1 Create a different display that does not use the 7-segment display.

N2 Implement all the stopwatch operations from the manual (elapsed time, net time and 1st–2nd place times).

N3 Implement the daily alarm.

N4 Implement the day and date function.

3 Analysis

This section refines the requirements and describes how each of them can be realised on the Zybo board.

3.1 Overall idea

The watch has to react to three things at the same time: button presses, switches and the passing of time. This can be done by using an interrupt for each of them. A hardware timer gives an interrupt every 10 ms and acts as the heart of the watch. This gives a resolution of 1/100 second, which is exactly what the stopwatch needs.

We keep the interrupt handlers short. They only read the inputs and set flags or counters. The slow printing to the serial port is done in the main loop. By not printing in the interrupts we do not block the timer interrupt, so no ticks are lost and the clock keeps the correct time.

3.2 Interrupts for buttons and switches

We use AXI GPIO blocks to connect the buttons and switches. They create an interrupt every time an input changes. Each interrupt gets its own handler. Two things must be kept in mind:

- An interrupt is also created when a button is released, so the handler must check which button is pressed. We use the release to know when button A is no longer held.
- Variables shared between the handlers and the main loop must be declared `volatile`.

3.3 Button bounce

Mechanical buttons bounce for a few milliseconds when pressed. Every bounce changes the input and can create an extra interrupt, so one press may be counted as several. This could be solved in software by ignoring new presses for a few timer ticks after a press, or in hardware with a debounce circuit. We decided not to implement debouncing at first and instead test whether it is a problem in practice (test B19 in Table 8).

3.4 ASCII graphics

Each digit is a small picture with a grid of 5 columns and 7 rows. We print a character where a segment is and a space where it is not. A terminal prints one line at a time, so we print row 0 of all the digits, then row 1 and so on.

To avoid a flickering or scrolling image we move the cursor to the top left corner and overwrite the old image. We only clear the screen when we change mode, and we only print when a shown value has changed.

3.5 Cycling between modes

The current mode is stored in one variable with four possible values: Time, Alarm, Stopwatch and Setting. This is the same order as on the real F-91W. Button C moves to the next mode and loops back to Time after the last mode. The LEDs show which mode the watch is in, and a small text at the top of the terminal also prints the current mode.

3.6 Setting the time

In Setting mode button L chooses which part of the time we change (seconds, minutes or hours). Button A adds one to the chosen value. The values wrap around when they pass their upper

limit (59 or 23). The clock does not count while we are in this mode, so the numbers do not change while the time is edited.

3.7 Quick digit advance

An interrupt only happens when a button changes, so the button interrupt alone can not tell that a button is still held. We solve this by letting the button handler remember if A is pressed or not. The 10 ms timer is then used to count how long A has been held. After 200 ticks (2 seconds) the value increases automatically every 10 ticks (10 times per second) until the button is released.

3.8 Showing the time and speeding it up

The clock counts 100 ticks for one second, 60 seconds for one minute and so on. To speed up the time we keep the timer period the same and instead update the clock several times per tick. If the switch is at 60x, the clock is updated 60 times per tick. If we changed the timer period instead, the stopwatch and the quick digit advance would also speed up, since they use the same timer.

3.9 Stopwatch with net time measurement

The stopwatch counts minutes, seconds and hundredths of a second. Button A starts and stops it. Net time measurement means that button L freezes the display while the stopwatch keeps counting in the background. Pressing L again shows the running time. This means that the counter and the display must be kept separate. When the stopwatch is stopped, L resets it. If the display is frozen when the stopwatch is stopped, the first L only releases the freeze and shows the final time. This gives the 1st–2nd place measurement from the manual.

3.10 Alarm

The alarm stores an hour, a minute and an on/off state. It must be compared with the clock time. At 60x speed one watch minute only lasts one real second, so the alarm must be checked after every clock update and not only once per tick. Otherwise it could be skipped at high speed. The alarm should start ringing once when the alarm minute begins, stop by itself when the minute is over, and stop when any button is pressed. Since the Zybo board has no buzzer, the alarm is shown by blinking the LEDs.

3.11 Day and date

When the clock passes midnight the date must move one day forward. The months have different lengths, and February has 29 days in leap years. A year is a leap year if it is divisible by 4, except years divisible by 100 that are not also divisible by 400. The weekday simply counts modulo 7. The watch starts on Thursday 1 January 2026, which is the correct weekday for that date.

4 Risk Analysis

Likelihood and consequence are rated from 1 (very unlikely / minor) to 5 (very likely / critical). The overall risk is the product of the two. The last column shows how we reduced each risk.

What can go wrong?	How likely is it to go wrong?	How serious are the consequences?	Overall risk	Mitigation
Timer period is wrong due to slow code	3	5	15	The timer handler only counts ticks. The main loop catches up with `handled_ticks`.
Serial port is too slow. The display flickers at 60x speed	3	5	15	Overwrite instead of clearing the screen, and only redraw when a value changes.
Button bounces and gives several presses for one press	2	3	6	Tested in practice (test B19).
Vivado bugs	3	4	12	Started from the given `zynq_interrupts` project and kept the work in git.
Interrupt not cleared	2	5	10	Every handler clears its interrupt flag before it returns.
Stopwatch freeze shows the wrong time	3	4	12	Counter and display are separated. A redraw is forced when the freeze is released.
Sickness in the group	1	4	4	Code shared in git, and both members know all parts.
Deadline	4	4	16	All need-to-have requirements first, nice-to-have afterwards.

Table 3: Risk analysis.

5 Design

This section describes how the solution is built, both the hardware and the software.

5.1 Hardware overview

We used Vivado to build the hardware. It consists of the Zynq processing system, three AXI GPIO blocks and one AXI timer, see Figure 1. The buttons, the switches and the timer can each raise an interrupt. The three interrupt lines are combined in a Concat block and go to the interrupt controller (GIC) in the processor through `IRQ_F2P`. The LEDs are only outputs. The full Vivado block design is shown in Appendix D.

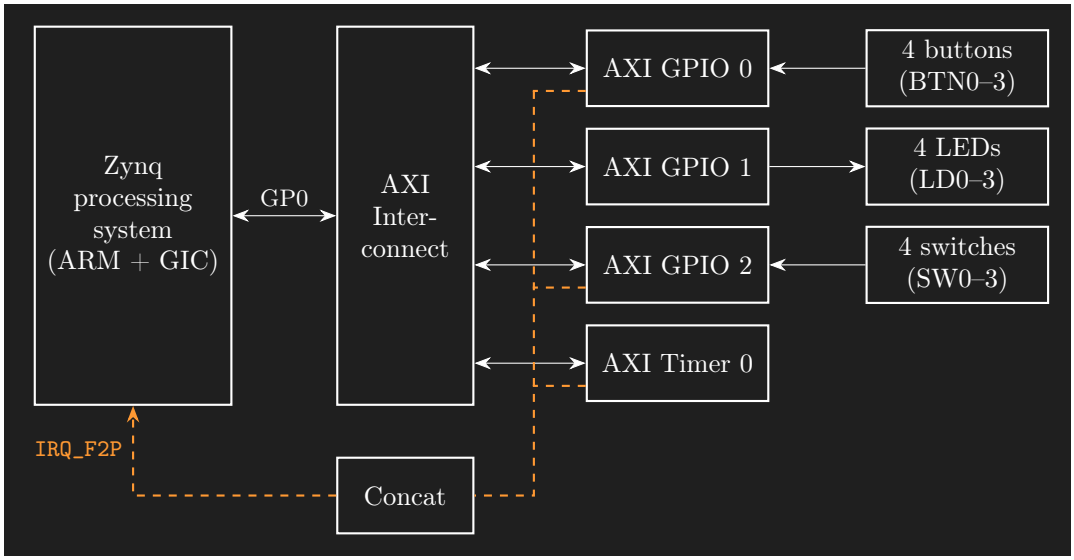


Figure 1: Simplified hardware block diagram. Dashed lines are interrupt lines.

5.2 Software structure

We split the project into five files: `Casio_main.c`, `clock.c`, `stopwatch.c`, `alarm.c` and `print.c`, see Figure 2. `Casio_main.c` is the only file that knows about the hardware. It handles the interrupts, the modes and the main loop. The other files only work with numbers and text, which also means they can be compiled and tested on a normal PC (Section 7).

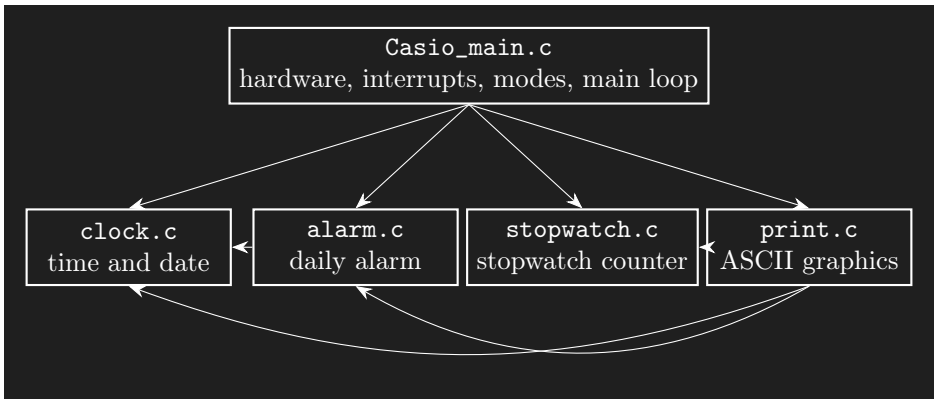


Figure 2: Software modules. An arrow means “uses”.

The main design idea is to keep the interrupts as fast as possible by only doing the essentials in them. The slow functions run in the main loop. The timer only counts ticks in `tick_count`.

The main loop counts how many ticks it has handled in `handled_ticks`. If printing takes longer than one tick, the main loop simply handles the missing ticks afterwards, so no clock ticks are lost.

5.3 Main loop

Figure 3 shows the main loop. First all missing ticks are handled: the clock and the alarm are updated `timerfactor` times, or in Setting mode the quick digit advance is handled instead. Then the stopwatch is updated if it is running. When all ticks are handled, the LEDs blink if the alarm is ringing, and the screen is redrawn if something has changed.

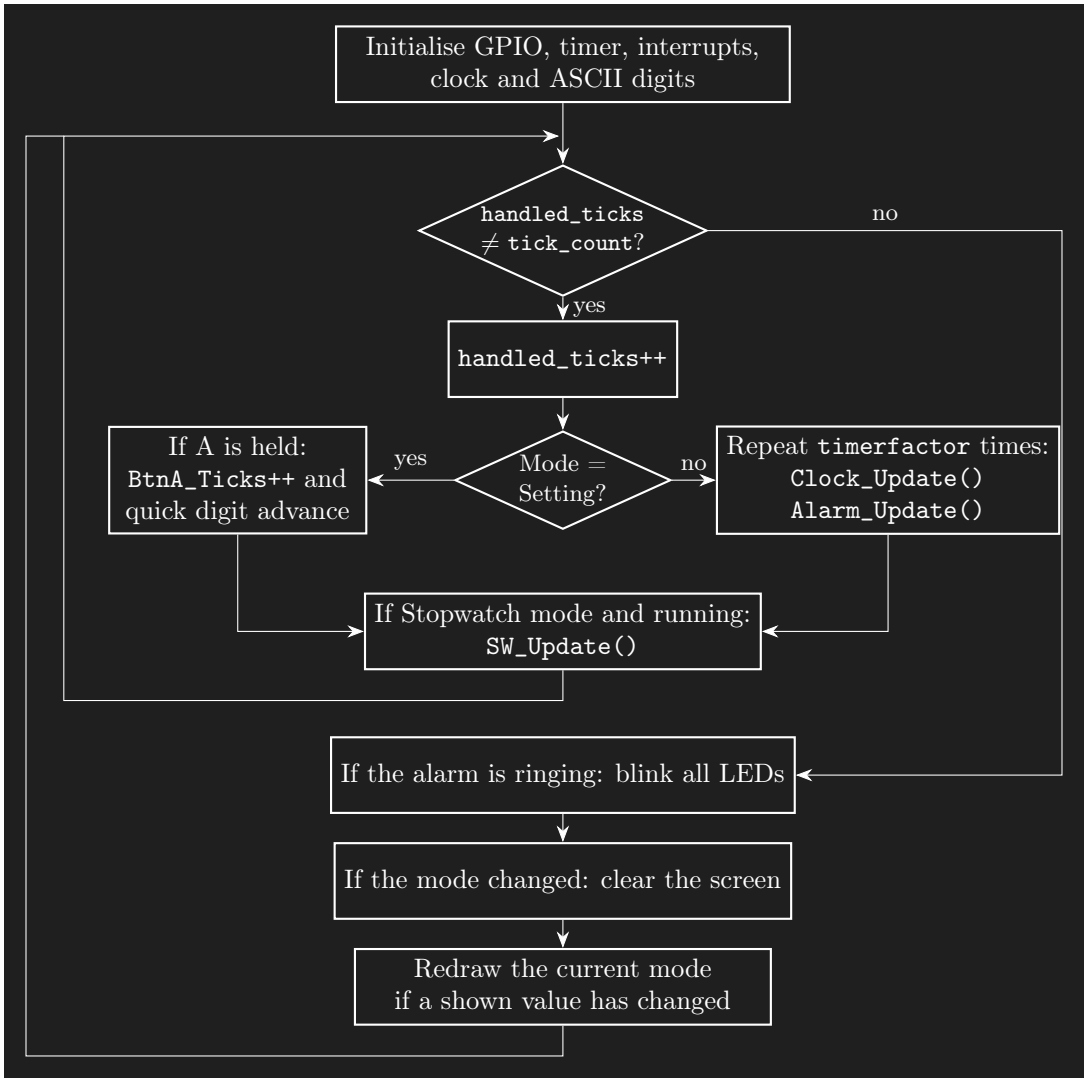


Figure 3: Flow diagram of the main loop.

5.4 Interrupts

The watch has three interrupts: buttons, switches and timer. Each of them has its own handler, and all three are connected to the interrupt controller (GIC) in the Zynq processor, see Figure 4.

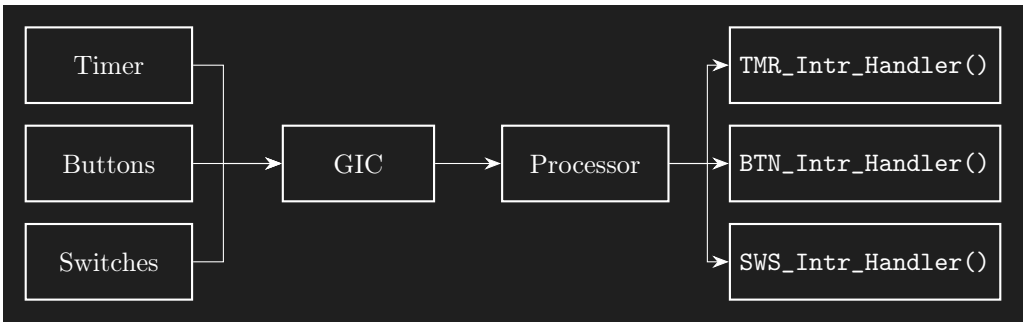


Figure 4: How the interrupts travel from the hardware to the handlers.

The order of how an interrupt travels:

1. A hardware block (timer or GPIO) raises its interrupt line.
2. The GIC forwards the interrupt to the processor, which pauses the main loop.
3. The processor jumps to the handler that is connected to that interrupt line.
4. The handler does its job and clears the interrupt flag in the hardware block.
5. The processor returns to the main loop exactly where it stopped.

The buttons are read as one 4-bit value. Table 4 shows which value each button gives and which watch button it emulates.

Board button	**Bit**	**Value**	**Watch button**
BTN0	0	1	A
BTN1	1	2	not used
BTN2	2	4	C (mode)
BTN3	3	8	L

Table 4: Button values.

5.5 Mode changer

Button C moves through the four modes in a loop, see Figure 5. The LEDs show the current mode. Table 5 shows what the buttons do in each mode.

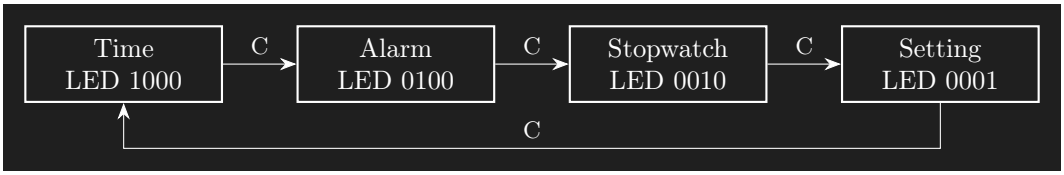


Figure 5: State diagram of the watch modes.

Mode	Button A	Button L	Button C
Time	nothing	nothing	next mode
Alarm	add one to the chosen value, or toggle on/off	choose hour → minutes → on/off	next mode
Stopwatch	start / stop	net time freeze / release / reset	next mode
Setting	add one to the chosen value (hold 2 s for quick advance)	choose seconds → minutes → hours	next mode
Alarm ringing	any button only stops the alarm		

Table 5: What the buttons do in each mode.

5.6 Stopwatch

The stopwatch is controlled by two flags: `StartStop_Bool` (is the counter running) and `SW_Show` (is the display updated). Together with the reset this gives the five states in Figure 6. The important point is that "frozen" only freezes the display and does not stop the counter.

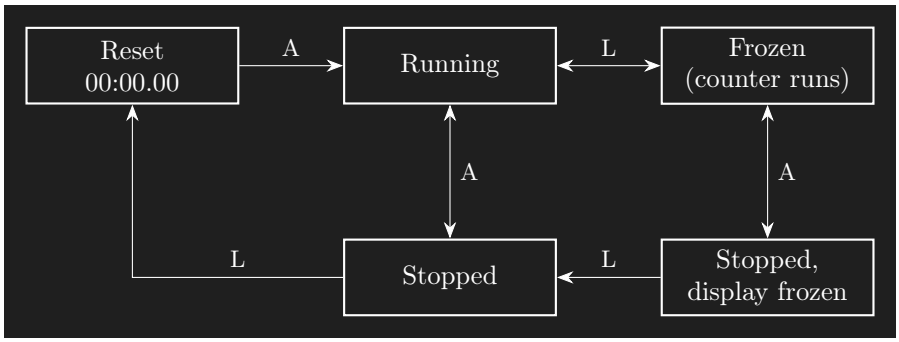


Figure 6: State diagram of the stopwatch.

Simplified:

- **Running:** the counter runs and the display is updated.
- **Frozen:** the counter runs, but the display is not updated (net time).
- **Stopped:** the counter is stopped. L resets the stopwatch to zero.
- **Stopped, display frozen:** the first L releases the freeze and shows the final time. This is used for the 1st–2nd place measurement.

5.7 Setting and quick digit advance

Button L chooses the value to change and button A adds one to it. Figure 7 shows the timing of the quick digit advance. The first step happens right away in the button interrupt. If A is still held after 200 ticks (2 s), the value advances every 10 ticks (0.1 s) until A is released.

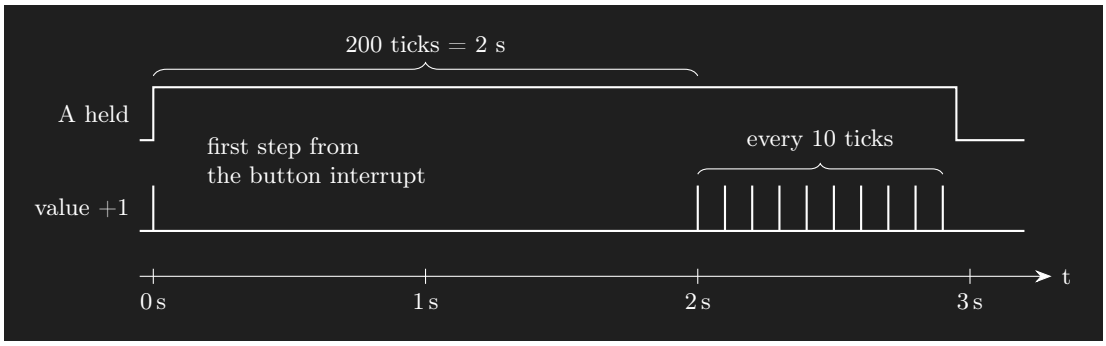


Figure 7: Timing of the quick digit advance.

5.8 Speed-up

The switches set a speed factor, and the clock is updated N times per tick. The highest switch that is on decides the factor, so the lower switches are “don’t care”, see Table 6.

SW3	SW2	SW1	SW0	Speed factor
1	–	–	–	60x
0	1	–	–	30x
0	0	1	–	20x
0	0	0	1	10x
0	0	0	0	1x

Table 6: Switch priority table. – means “don’t care”.

The clock keeps running in the Alarm and Stopwatch modes. It only stops in Setting mode, so the numbers do not change while the user edits the time. The stopwatch is not affected by the speed switches.

5.9 Alarm

The alarm has three values that can be edited: hour, minutes and on/off. L chooses which value to edit and A changes it. After every clock update the alarm is compared with the clock. A flag remembers that the alarm has already started in the current minute, so it only starts ringing once. When the minute is over, the alarm stops by itself and the flag is cleared, so it is ready for the next day.

5.10 Day and date

The date is part of the clock. When the hour passes 23, the clock calls a function that moves the date one day forward. It uses a table with the number of days in each month plus the leap year rule from the analysis. The weekday is an index from 0 to 6 into a list of weekday names.

5.11 Display design

Each digit is stored as 7 text lines of 5 characters. The hours and minutes use the big digits, and the seconds or hundredths of a second use the small digits. To draw a time, the program prints line 0 of all the digits, then line 1 and so on. Figure 8 shows the Time mode as it looks in the terminal.

```
Mode: Time

Thu 01/01/2026
  0   00000        00000 0    0
 00        0  0 0       0 0    0
  0        0  0 0       0 0    0  000    000
  0       0           00  00000 0      0
  0     0      0 0       0      0  000    000
  0   0        0 0       0      0      0 0    0
00000 00000        00000      0  000    000
```

Figure 8: Time mode showing 12:34:56. The seconds are drawn with the small digits.

We use three ANSI escape codes to control the terminal (Table 7). By moving the cursor to the top-left corner we overwrite the old text with the new text and avoid flickering. The screen is only cleared when the mode changes.

Escape code	**Effect**
`\033[2J`	clear the whole screen
`\033[H`	move the cursor to the top-left corner
`\033[?25l`	hide the cursor

Table 7: ANSI escape codes used.

6 Implementation

This section gives an overview of the code and the most important parts. We split the code into different files to make it easier for us to work on separate parts.

6.1 Hardware setup and interrupts

`main()` initialises the three GPIO blocks and the AXI timer. The LEDs are set as outputs and the buttons and switches as inputs. The timer counts down, reloads by itself and gives an interrupt each time it reaches zero. The reload value `TMR_LOAD` is 1 000 000, which is 10 ms with a 100 MHz clock.

The interrupt controller is set up in `IntcInitFunction()` in the order described in the Design section. The three handlers are `TMR_Intr_Handler()`, `BTN_Intr_Handler()` and `SWS_Intr_Handler()`. The timer handler only does one thing: it increments `tick_count` and clears the interrupt flag. All variables shared between the handlers and the main loop are declared `volatile`.

6.2 Main loop

The main loop follows Figure 3. The inner `while (handled_ticks != tick_count)` loop makes sure that every timer tick is handled exactly once, even if the printing was slow. Each mode remembers the last value it printed (for example `last_seconds` in Time mode) and only redraws when the value has changed, or when `need_redraw` or one of the print flags is set by a button.

6.3 Button interrupts

`BTN_Intr_Handler()` disables the button interrupt and checks that the interrupt came from the button channel. It then reads the button value, acts on it, clears the interrupt and enables it again. The values are shown in Table 4.

Releasing a button also gives an interrupt, but then the value is 0, so it matches no case and nothing happens. The release is still used for the quick digit advance: `BtnA_Held` is set from bit 0 of the value, and when A is released the hold counter `BtnA_Ticks` is reset to 0. If the alarm is ringing, any button press stops it, and the value is set to 0 so the button does not do its normal action.

At the end of the function `Show_Mode_LED()` sets the LEDs from `current_mode`, so they always match the mode. When the mode changes, the handler only sets the flag `btn_changed`. The main loop then clears the screen.

6.4 Switch interrupt

`SWS_Intr_Handler()` reads the four switches and sets `timerfactor`. It uses a simple `if / else if` chain. It starts with SW3, and only the highest switch that is on matters. This gives the priority in Table 6.

6.5 Clock, setting and quick digit advance

`Clock_Update()` adds 10 ms to a counter. At 1000 ms it adds one second, and the overflow is passed on to minutes, hours and the date. The three setting functions `Clock_SetSeconds()`, `Clock_SetMinutes()` and `Clock_SetHour()` add one to their value and use modulo 60 or 24, so the value wraps around.

`Setting_Step()` calls the right setting function depending on `Chance_setting`, which button L changes. A press on A calls `Setting_Step()` once from the button interrupt. The quick digit advance is done in the main loop: in Setting mode, while `BtnA_Held` is set, `BtnA_Ticks` is increased for every tick. When it reaches 200, and then every 10 ticks after that, `Setting_Step()` is called.

6.6 Stopwatch

The stopwatch has its own counter in `stopwatch.c`. `SW_Update()` adds 10 ms per tick, and when 1000 ms is reached it adds one second. Minutes and seconds are calculated from the total number of seconds, so the stopwatch wraps from 59:59.99 to 00:00.00.

The net time measurement uses the flag `SW_Show`. It only decides whether the screen is updated while the counter keeps running. While running, the screen is updated every 50 ms (20 times per second), which prevents the processor from being flooded with prints. Button L is also handled in `BTN_Intr_Handler()` and depends on the state of the stopwatch described in Figure 6. When the freeze is released or the stopwatch is stopped, `Print_Flag` is set. This forces a redraw, so the screen shows the exact time and not only the last 50 ms step.

6.7 ASCII graphics

The digits are stored in `print.c` as two arrays, `bigChars` and `smallChars`, each with 12 characters: the digits 0–9, a colon and a space. Each character is a struct `AsciiDigit` with 7 lines of 5 characters. The lines are allocated and filled once by `printAsciiDigit()`, which is called from `Print_Init()` at start-up.

`print_digit()` takes a string such as `"12:3456"`. The first characters are drawn with the big digits and the last two with the small digits. A character is converted to an array index by subtracting `'0'`. Since `':'` comes right after `'9'` in the ASCII table, the colon gives index 10, which is exactly where the colon is stored. `Print_Clock()`, `Print_Stopwatch()` and `Print_Alarm()` build the string with `sprintf()` and call the drawing code.

6.8 Alarm

The alarm is implemented in `alarm.c`. `alarmSetting` selects the value to edit (0 = hour, 1 = minutes, 2 = on/off). `Alarm_Advance()` is called by L and moves to the next value, and `Alarm_AppendOne()` is called by A and adds one to the hour or minutes or toggles the alarm on or off.

`Alarm_Update()` is called right after every `Clock_Update()`, also inside the speed-up loop, so the alarm can not be skipped at high speed. `Alarm_Check()` is true when the alarm is on and the hour and minute match the clock. The first time it is true, `alarmTriggered` and `alarmRinging` are set. When the minute is over, both are cleared again. `Alarm_Stop()` only clears `alarmRinging`, so a stopped alarm does not start again in the same minute. While the alarm is ringing, the main loop blinks all four LEDs every 50 ticks (0.5 s). When it stops, the mode LED is shown again.

6.9 Day and date

The date is implemented in `clock.c`. When the hour wraps from 23 to 0, `Clock_NextDay()` is called. It moves the weekday one step forward modulo 7 and increments the day. If the day is larger than `Clock_DaysInMonth()`, the month is incremented, and after December the year is incremented. `Clock_DaysInMonth()` uses the table `daysInMonth` and returns 29 for February in

leap years. The watch starts on 1 January 2026, which is a Thursday, so the weekday list starts with `"Thu"`. `Print_Date()` prints the date above the time in Time mode.

7 Test

7.1 Test method

We tested in two ways:

- **Board tests (B):** the program runs on the Zybo board and we press the buttons and switches and watch the terminal and the LEDs.
- **PC unit tests (P):** `clock.c`, `alarm.c` and `stopwatch.c` do not use the hardware, so they can be compiled on a PC together with a small test program (`tests/test_logic.c`). This lets us test things that take too long on the board, such as month and leap year changes. The full test report is in Appendix C.

7.2 Test summary

Table 8: Test summary.

#	Req.	Test case	Expected result	Actual result	Remarks
B1	R3	Start the program	Time mode is shown, LEDs show 1000	As expected	
B2	R3	Press C four times	Time → Alarm → Stopwatch → Setting → Time, LEDs 1000, 0100, 0010, 0001	As expected	The screen is cleared at every mode change
B3	R6	Let the clock run for minutes and compare with a phone clock	Seconds count in real time	As expected	
B4	R6	Turn on SW0, SW1, SW2, SW3 one at a time	Speed 10x, 20x, 30x, 60x	As expected	At 60x the time is correct, but the display flickers a little and skips some seconds (Section 8.3)
B5	R6	Turn on several switches	The highest switch decides the speed	As expected	Matches Table 6
B6	R2	Time mode display	Static image without scrolling or flicker	As expected	A little flicker only when changing mode
B7	R4	Setting mode: L, then A	L selects seconds → minutes → hours, A adds one	As expected	The display does not show which value is selected (bug B4)
B8	R4	Stay in Setting mode for 10 seconds	The time does not change while editing	As expected	The clock is paused in this mode by design
B9	R5	Hold A for more than 2 s in Setting mode	After 2 s the value advances quickly	As expected	
B10	R5	Release A during quick advance	The value stops advancing	As expected	The hold counter is reset on release
B11	R7	Stopwatch: press A, then A again	Starts, then stops	As expected	

#	Req.	Test case	Expected result	Actual result	Remarks
B12	R7	Stopwatch: press L while running, then L again	Display freezes while counting continues, then jumps to the running time	As expected	Net time measurement
B13	R7	Stopwatch: press L when stopped	The stopwatch is reset to 00:00.00	As expected	If the display is frozen, the first press only releases the freeze
B14	R7	Stopwatch running with a speed switch on	The stopwatch is not affected by the switches	As expected	By design
B15	N2	Start the stopwatch, change to Time mode, go back	The stopwatch has kept running in the background	**Fails**	`SW_Update()` is only called in Stopwatch mode (bug B1)
B16	R1	Press two buttons at the same time	One of the buttons is handled	Both are ignored	The `switch` uses the whole button value (bug B2)
B17	N3	Set the alarm one minute ahead and wait	LEDs blink at the alarm time, the rest of the watch keeps running	As expected	
B18	N4	Day and date in Time mode	Day and date are shown and updated	As expected	The date can not be set (bug B5)
B19	R1	Press A 20 times in Setting mode	The value increases by exactly 20	As expected	Checks for button bounce
P1	R6	100 clock updates	00:00:01	Pass	
P2	N4	23:59:59 + 10 ms	00:00:00, Thu 01/01 → Fri 02/01	Pass	
P3	N4	Month and year change	31/01 → 01/02, 28/02/2026 → 01/03, 31/12/2026 → Fri 01/01/2027	Pass	1 Jan 2027 is a Friday
P4	N4	Leap year 2028	28/02 → 29/02 → 01/03	Pass	
P5	R4	Setting wrap-around	59 → 0 for seconds and minutes, 23 → 0 for hours	Pass	
P6	N3	Alarm 00:01 at 60x, stop with button, alarm off	Rings exactly once and stops after the minute; no new ring after stop; no ring when off	Pass	
P7	R7	Stopwatch counting and wrap	6123 ticks = 01:01.23; 59:59.99 → 00:00.00	Pass	

8 Discussion

In this section we look at how well the solution meets the requirements, which design choices worked well and what could be done better.

8.1 Fulfilment of the requirements

Table 9 shows the status of each requirement. All seven need-to-have requirements are fulfilled. Of the nice-to-have requirements, the alarm is fully implemented, and the stopwatch and day and date are implemented with small limitations. The different display style (N1) was not implemented.

ID	Requirement	Status	Comment
R1	Interrupts	Fulfilled	Two buttons at once are ignored (B16)
R2	ASCII graphics, static image	Fulfilled	Small flicker at 60x
R3	Cycle between modes	Fulfilled	B1, B2
R4	Set the time	Fulfilled	B7, B8, P5
R5	Quick digit advance	Fulfilled	B9, B10
R6	Show time and speed-up	Fulfilled	B3–B5, P1
R7	Stopwatch with net time	Fulfilled	B11–B14, P7
N1	Different display	Not done	
N2	All stopwatch operations	Partly	All operations work, but the stopwatch stops when the mode is left (B15)
N3	Alarm	Fulfilled	B17, P6
N4	Day and date	Partly	Counts correctly (P2–P4), but can not be set

Table 9: Fulfilment of the requirements.

8.2 What worked well

Short interrupt handlers. The timer handler only counts ticks, and all printing is done in the main loop. This keeps the time precise. As extra insurance the main loop counts the ticks it has handled in `handled_ticks`, so if printing takes longer than a tick, the missing ticks are handled afterwards. The button handler does a bit more work than the others, since it changes modes and calls the stopwatch, alarm and setting functions directly. These are only a few assignments, so they are still fast.

The speed-up. We keep the hardware timer fixed at 10 ms and update the clock several times per tick in software instead. This made it easy to change, since only one variable is needed, and it means that the stopwatch and the quick digit advance are not affected.

Separating the display from the counters. This makes it possible to show one mode while the clock keeps running, and it made the full stopwatch easy to build with a few flags. It also avoids overloading the serial port, since the display is only redrawn when a value changes.

Splitting the code into files. This made the project easier to manage and to work on in parallel. Only the main file knows about the hardware, so the clock, alarm and stopwatch could be tested on a PC.

8.3 Limitations

Serial port speed. A redraw of the Time mode sends about 350 bytes. At the default 115 200 baud, with 10 bits per byte, this takes about 30 ms, so the terminal can show at most about 33 redraws per second. At 60x the seconds change 60 times per real second, so the display can not show every second and flickers a little. The time itself stays correct, because the ticks are

caught up afterwards. This could be improved by only redrawing the digits that have changed or by using a higher baud rate.

Buttons. The buttons are read as one value, so pressing two buttons at the same time is ignored. There is no software debouncing.

Interrupt priority. We use the default GIC priorities for all three interrupts. This is fine because all handlers are short, but the GIC could give the timer the highest priority.

Shared variables. The variables inside `clock.c` and `alarm.c` are changed both from the button interrupt (setting) and from the main loop (updating), and they are not `volatile`. This is mostly safe because the clock is not updated in Setting mode, but a cleaner solution would be to only set flags in the interrupt and let the main loop change the values.

Timer accuracy. According to the AXI Timer documentation, the period in down-count mode is `TMR_LOAD` + 2 clock cycles, so the real period is 10.00002 ms instead of 10 ms. This makes the clock about 0.2 s slow per day, which is negligible for this project.

8.4 Comparison with the real Casio F-91W

The emulator covers the main functions of the F-91W, but it differs in some ways:

- The clock is paused in Setting mode. This is good for editing, but the watch loses the time spent in this mode.
- On the real watch A resets the seconds to 00 in Setting mode. In our emulator it adds one.
- The real watch blinks the value that is being set. Our display does not show which value is selected.
- The date can not be set.
- The stopwatch stops when the mode is left. On the real watch it keeps running.
- There is no 12/24-hour format, no hourly time signal and no light.

8.5 Future work

The most important improvement is to let the stopwatch run in the background (bug B1), which is a one-line change. After that we would show the selected value in Setting mode, make the date settable, add software debouncing and only redraw the parts of the screen that change.

9 Conclusion

We have built an emulator of the Casio F-91W digital watch on the Zybo board. All seven mandatory requirements are implemented and tested: interrupts for buttons and switches, ASCII graphics with a static image, cycling between the modes, setting the time, quick digit advance, showing the time with speed-up, and the stopwatch with net time measurement. We also implemented three of the optional extensions: the full stopwatch operations, the alarm and the day and date. Two of them have known limitations: the stopwatch stops when the mode is left, and the date can not be set.

The solution is built around a 10 ms timer interrupt and short interrupt handlers. All printing is done in the main loop, and missing ticks are caught up, so the timekeeping stays correct. Logic that does not use the hardware was also verified with unit tests on a PC. The main limitation is the serial port, which can not redraw the screen fast enough at 60x speed.

A User manual

Getting started

1. Connect the Zybo board to the PC with the USB cable (PROG/UART port) and turn it on.
2. Program the FPGA with the bitstream and run the application from Vitis.
3. Open a serial terminal that supports ANSI escape codes (for example PuTTY or Tera Term) on the board's COM port with 115 200 baud, 8 data bits, no parity and 1 stop bit.
4. The watch starts in Time mode at 00:00:00 on Thursday 01/01/2026.

Buttons, switches and LEDs

Board	Function
BTN0	Button A
BTN2	Button C (change mode)
BTN3	Button L
SW0 / SW1 / SW2 / SW3	Speed 10x / 20x / 30x / 60x (highest switch wins)
LD3 / LD2 / LD1 / LD0	Time / Alarm / Stopwatch / Setting mode
All LEDs blinking	The alarm is ringing

Using the watch

Change mode: press C. The order is Time → Alarm → Stopwatch → Setting.

Set the time: go to Setting mode. Press L to choose seconds, minutes or hours. Press A to add one. Hold A for more than 2 seconds to advance quickly. Press C to leave.

Set the alarm: go to Alarm mode. Press L to choose hour, minutes or on/off. Press A to add one or to turn the alarm on and off. The bottom line shows whether the alarm is on and what is selected.

Stop the alarm: press any button. The alarm also stops by itself after one minute.

Stopwatch: go to Stopwatch mode. A starts and stops. While running, L freezes the display (net time), and L again shows the running time. When stopped, L resets to 00:00.00.

1st–2nd place times: press A to start. When the first runner finishes, press L (the display freezes). When the second runner finishes, press A. Read the first time, press L to see the second time, and press L again to reset.

B Bug report

ID	Description	Cause	Possible fix	Severity
B1	The stopwatch stops when the mode is changed (test B15)	`SW_Update()` is only called in Stopwatch mode (`Casio_main.c`, line 371)	Remove the mode check in the main loop, so only `StartStop_Bool` decides	Medium
B2	Two buttons pressed at the same time are ignored (test B16)	The `switch` statement compares the whole button value	Check each bit of the button value on its own	Low
B3	No software debouncing	Not implemented	Ignore new presses for 2–3 ticks after a press	Low (see B19)
B4	Setting mode does not show which value is selected	Not implemented	Print the selected value as in Alarm mode, or blink it	Low
B5	The date can not be set	Not implemented	Add day, month and year to Setting mode	Low
B6	At 60x the display flickers and skips seconds	One redraw (≈350 bytes) takes ≈30 ms at 115 200 baud	Only redraw the changed digits, or use a higher baud rate	Low
B7	Quick digit advance does not work in Alarm mode	The hold counter is only used in Setting mode	Also use `BtnA_Ticks` in Alarm mode	Low

C Test reports

PC unit tests

The file `tests/test_logic.c` tests `clock.c`, `alarm.c` and `stopwatch.c` without the hardware. The test script is written entirely by Claude Code but validated by us. Each call to `Clock_Update()` or `SW_Update()` is one 10 ms tick. It is compiled and run from the project folder with

```
cc -O2 -I. -o test_logic tests/test_logic.c clock.c alarm.c stopwatch.c
./test_logic
```

Output:

```
100 ticks = 00:00:01                                    PASS
1 tick before midnight = 23:59:59, Thu 01/01/2026       PASS
23:59:59 -> 00:00:00 Fri 02/01/2026                     PASS
31/01/2026 -> 01/02/2026                                PASS
28/02/2026 -> 01/03/2026 (not leap year)                PASS
31/12/2026 -> 01/01/2027 is a Friday                    PASS
28/02/2028 -> 29/02/2028 (leap year)                    PASS
29/02/2028 -> 01/03/2028                                PASS
Set seconds 60 times wraps 59 -> 0                      PASS
Set minutes 59 times = 59                               PASS
Set minutes once more wraps 59 -> 0                     PASS
Set hour 24 times wraps 23 -> 0                         PASS
Alarm 00:01 rings at 60x speed                          PASS
Alarm starts ringing exactly once                       PASS
Alarm stopped by itself after the minute                PASS
Alarm stopped by button does not ring again             PASS
Alarm OFF does not ring                                 PASS
Stopwatch 6123 ticks = 01:01.23                         PASS
Stopwatch shows 59:59.99                                PASS
Stopwatch wraps 59:59.99 -> 00:00.00                    PASS

0 failure(s)
```

Board tests

The board tests B1–B19 were done by hand with the terminal and the LEDs as described in Table 8.

D Extra diagrams

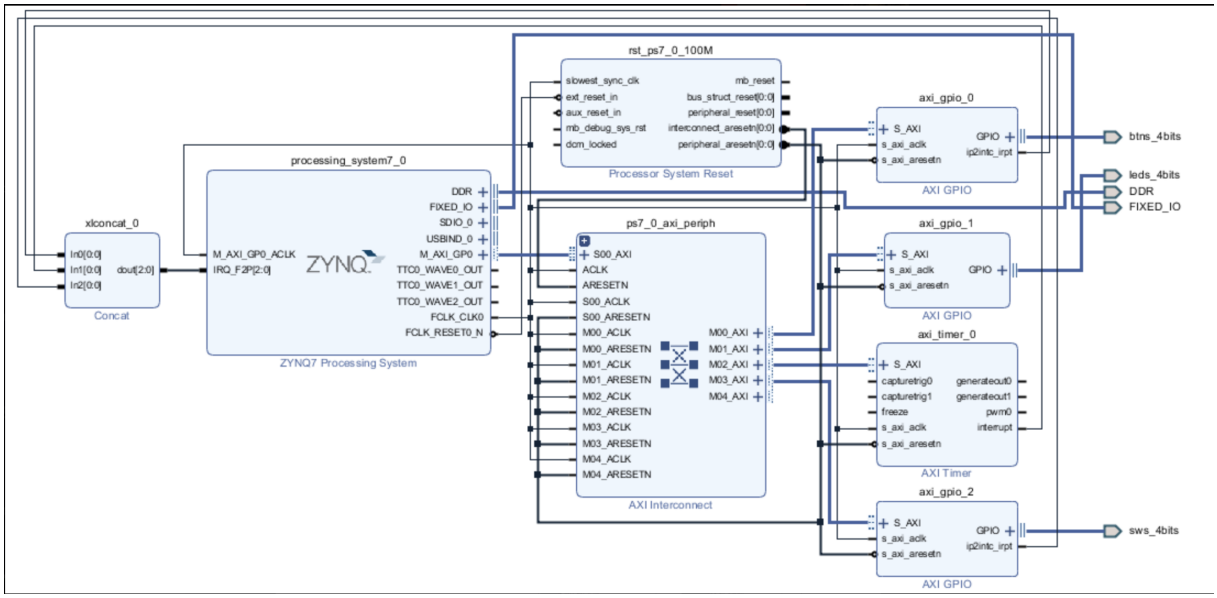


Figure 9: Full Vivado block design with the Zynq processor, the three AXI GPIO blocks, the AXI timer and the Concat block for the interrupts.